Pub/Sub with MQTT

Redes e Serviços

Daniel Corujo, Paulo Salvador, Miguel Matos

7 May 2026

Foreword

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe protocol that runs over TCP. Publishers send messages tagged with a *topic*; subscribers declare interest in topics; a central **broker** routes each message to every matching subscriber. Publishers and subscribers never connect to each other directly; all traffic flows through the broker. This decoupling means adding a new consumer requires no change to the publisher.

In this guide, the broker is **Mosquitto**, running as a Docker container. Python clients use the **paho-mqtt** library.

Exercises

Practical Lab: Pub/Sub with MQTT

Objective: Run a local Mosquitto broker, write Python publisher and subscriber clients, observe MQTT topic routing, and understand how QoS levels, retained messages, and Last Will affect delivery semantics.

All exercises run on the class VM. The broker runs in Docker; Python scripts run directly on the VM.

Setup

Start the broker from the support files directory available on the e-Learning platform:

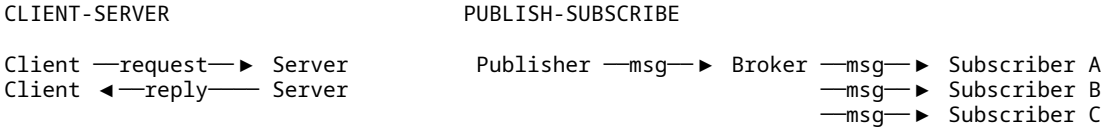
```
$ docker compose up -d
```

This starts Mosquitto on `localhost:1883`. Install the CLI diagnostic tools and the Python library:

```
$ sudo apt install mosquitto-clients
$ python3 -m venv venv && source venv/bin/activate
$ pip install "paho-mqtt==1.6.1"
```

Part 1: First Connection (Publisher and Subscriber)

In every exercise covered so far (UDP datagrams, TCP echo, the chat server), the sender always needed to know the address of the receiver. Pub/sub removes that requirement. A publisher sends to a **topic string**; a subscriber declares interest in a topic string; the broker handles the routing. Neither side knows the other exists.



The paho-mqtt library is callback-driven. You register `on_connect` and `on_message` functions; the library calls them from a background network thread. You do not write a `recv` loop.

1.1 Verifying the Broker with CLI Tools

Open two terminals. In terminal 1, subscribe to a topic:

```
$ mosquitto_sub -h localhost -t "rs/lab" -v
```

In terminal 2, publish a message:

```
$ mosquitto_pub -h localhost -t "rs/lab" -m "hello from terminal"
```

The message appears in terminal 1 immediately. Open a third terminal and run a second `mosquitto_sub` on the same topic. Publish again. Both subscribers receive it simultaneously.

> **Thinking Question:** In the TCP chat server from the previous class, the server maintained a dictionary mapping channel names to connected client sockets. Where does that bookkeeping live in MQTT? If the broker crashes and restarts, what happens to active subscriptions?
>
> **Thinking Question:** Stop `mosquitto_sub` in terminal 1. Publish five messages from terminal 2. Restart `mosquitto_sub` and subscribe again. Did you receive any of the five messages? Compare this to what would happen in the TCP chat server's channel model.

1.2 Publisher and Subscriber scripts

Download the `publisher.py` and `subscriber.py` scripts from the support files available on the e-Learning platform. The publisher reads lines from `stdin` and publishes each one to topic `rs/lab09/chat`; the subscriber listens on the same topic and prints every message it receives.

`loop_start()` runs the network loop in a background thread, leaving the main thread free for `input()`. On the subscriber side, `subscribe()` is called inside `on_connect` (not before `connect()`) and `loop_forever()` blocks the main thread while the library calls `on_message` for each arriving message.

Run `publisher.py` in one terminal and `subscriber.py` in another. Start a second `subscriber.py` in a third terminal. All messages appear in both subscribers simultaneously.

> **Thinking Question:** The subscriber calls `client.subscribe()` inside the `on_connect` callback. What could go wrong if you called `subscribe()` immediately after `connect()`, before the connection completes? What happens to subscriptions when a client disconnects and reconnects?
>
> **Thinking Question:** `msg.payload` is of type `bytes`, not `str`. You must call `.decode()` to get a string. Why does paho-mqtt deliver the payload as bytes? What types of data might an MQTT publisher send that are not valid UTF-8?

Part 2: Topic Hierarchies and Wildcards

Topics are hierarchical UTF-8 strings, with levels separated by `/`. They are not pre-declared: any publisher can publish to any topic string, and the broker creates or destroys topic state dynamically. Two wildcard characters allow a single subscription to match multiple topics:

- **+** matches exactly one topic level. `home/+/temperature` matches `home/kitchen/temperature` but not `home/kitchen/oven/temperature`.
- **#** matches zero or more remaining levels and must appear last. `home/#` matches `home/kitchen`, `home/kitchen/temperature/raw`, and any other topic under `home/`.

When a publisher sends a message, the broker checks every active subscription for a topic match and delivers a copy to each matching subscriber.

2.1 Simulating a Multi-Room Sensor Network

Use `sensor_sim.py` from the support files available on the e-Learning platform. It publishes simulated sensor readings every 2 seconds.

The topic hierarchy looks like this:

```
home/
├── kitchen/
│   ├── temperature
│   └── humidity
├── bedroom/
│   └── temperature
└── living_room/
    ├── temperature
    └── co2
```

Run the simulator and observe all messages with:

```
$ mosquitto_sub -h localhost -t "home/#" -v
```

Then try narrower filters:

```
$ mosquitto_sub -h localhost -t "home/kitchen/#" -v
$ mosquitto_sub -h localhost -t "home/+/temperature" -v
```

2.2 Selective Subscription with Wildcards

Use `dashboard.py` from the support files available on the e-Learning platform. It subscribes only to temperature readings across all rooms.

Run `dashboard.py` alongside `sensor_sim.py`. Confirm that it receives temperature readings from kitchen, bedroom, and living room, but not humidity or CO2.

> **Thinking Question:** Without modifying `dashboard.py`, add a sensor that publishes to `home/attic/temperature` in the sensor simulator.. Does the dashboard automatically receive its readings? What change would have been required in the TCP chat server from the previous guide to achieve the same result?

2.3 Multi-Level Wildcards

Publish to a topic deeper than the simulator uses:

```
$ mosquitto_pub -h localhost -t "home/kitchen/oven/temperature" -m "210.0"
```

Run two subscribers simultaneously: one with `home/+/temperature` and one with `home/#`. Observe which one receives the message.

`home/+/temperature` matches exactly the pattern `home/<one-level>/temperature`. The path `home/kitchen/oven/temperature` has four levels, so `+` cannot match `kitchen/oven`. Only `home/#` receives it.

> **Thinking Question:** The # wildcard must be the last character in a subscription filter. Why does the protocol enforce this restriction? What would be ambiguous or computationally expensive if # could appear in the middle of a filter?

Part 3: QoS Levels

The MQTT protocol defines three quality-of-service levels that control the delivery guarantee between a client and the broker:

L	Name	Guarantee	Packet exchange
0	At most once	Fire and forget; no acknowledgement	`PUBLISH` only
1	At least once	Broker acknowledges; publisher retransmits if no ACK	`PUBLISH` -> `PUBACK`
2	Exactly once	Four-step handshake; no duplicates, no loss	`PUBLISH` -> `PUBREC` -> `PUBREL` -> `PUBCOMP`

QoS is specified independently by the publisher (on each `publish()` call) and by the subscriber (on each `subscribe()` call). The effective QoS delivered to a subscriber is the minimum of the two.

3.1 QoS 0 - Fire and Forget

Start the publisher and the subscriber in two different terminals. They both default to QoS level 0.

```
$ python3 publisher.py
$ python3 subscriber.py
```

Send some messages from the publisher. You should see them arrive at the subscriber, since it is connected to the broker.

Interrupt the subscriber, and send some messages while the subscriber is offline. Then, start it again.

```
$ python3 subscriber.py
```

No messages arrive, since, for QoS 0 messages, the broker either delivers them at the moment of publish or drops them.

> **Thinking Question:** MQTT runs over TCP, which retransmits lost packets automatically. Why is TCP's retransmission not enough to guarantee message delivery?

3.2 QoS 1 - At Least Once

Restart the publisher with a higher QoS level:

```
$ python3 publisher.py --qos 1
```

Then, restart your subscriber, with a set client id, a persistent session, and QoS level 1.

```
$ python3 subscriber.py --client-id sub-01 --qos 1 --persistent
```

Wait for `Connected`, then interrupt it with `Ctrl+C`.

Publish some messages at QoS 1 while the subscriber is offline. Then restart the subscriber with the same identity and QoS:

```
$ python3 subscriber.py --client-id sub-01 --qos 1 --persistent
```

All messages now arrive. The broker buffered them for the persistent session and flushes the queue on reconnect.

Now look carefully at the output: in theory, some messages could appear twice. If the TCP connection drops after the broker sends PUBACK but before the publisher receives it, the publisher retransmits the `PUBLISH`. The broker has no way to know the first delivery succeeded, so it delivers the message again. This is the "at least once" trade-off.

> **Thinking Question:** A subscriber receives the same temperature reading twice: `{"sensor": "kitchen", "temp": 22.4}`. Is this a problem for a dashboard displaying the current reading? Now consider receiving `{"command": "unlock_door", "id": 42}` twice. What would happen if the subscriber processed this message without checking for duplicates? How does this motivate QoS 2?

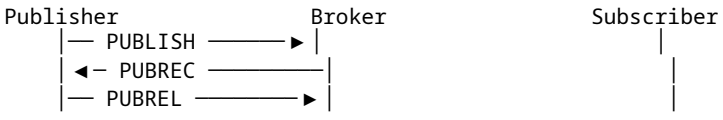
3.3 QoS 2 - Exactly Once

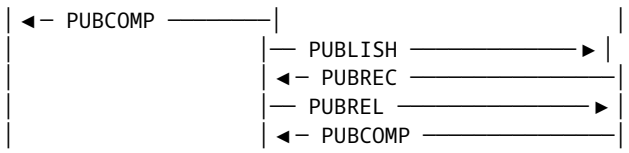
Start a Wireshark capture on the loopback interface with the display filter `tcp.port == 1883`.

Publish a single message at QoS 2:

```
$ mosquitto_pub -h localhost -t "rs/test" -m "critical" -q 2
```

Stop the capture and follow the TCP stream. Identify the four MQTT control packets that make up the QoS 2 handshake:





In the raw bytes view, the MQTT message type is encoded in the upper 4 bits of the first byte: PUBLISH = 3, PUBREC = 5, PUBREL = 6, PUBCOMP = 7.

> **Thinking Question:** QoS 2 requires four packets to deliver one message, compared to one packet for QoS 0. Under what conditions is this overhead worthwhile? For a sensor network reporting temperature every 5 seconds, which QoS would you choose for the temperature readings? Which for an "activate emergency shutdown" command?

Part 4: Retained Messages and Last Will

4.1 Retained Messages

Normally, a subscriber only receives messages published *after* it subscribes. A **retained message** changes this: the broker stores the last message published to a topic with the retain flag set, and delivers it immediately to any new subscriber, even if the publisher went offline hours ago.

Publish a retained message:

```
$ mosquitto_pub -h localhost -t "home/boiler/status" -m "ON" -r
```

Wait 30 seconds. Subscribe to that topic:

```
$ mosquitto_sub -h localhost -t "home/boiler/status" -v
```

The message `ON` arrives immediately, even though the publisher has long since exited. Only one retained message is stored per topic; publishing a new one replaces it. Clear a retained message by publishing an empty payload:

```
$ mosquitto_pub -h localhost -t "home/boiler/status" -m "" -r
```

Subscribe again. Nothing arrives.

> **Thinking Question:** A dashboard subscribes to `home/+/status` to display the current state of all devices. Without retained messages, the dashboard shows nothing until each device next publishes. With retained messages, the dashboard populates instantly on startup. What is the cost of this feature at scale? The broker must store one message per retained topic. What happens if 10,000 devices each retain a status message?

4.2 Last Will and Testament

When a client connects to MQTT, it can register a **Last Will and Testament (LWT)**: a topic, payload, QoS, and retain flag. If the client disconnects unexpectedly (that is, the TCP connection drops without a `DISCONNECT` packet), the broker publishes the will message on the client's behalf. A clean disconnect suppresses the will.

This enables presence detection: a device publishes `online` at connect time and registers a will of `lost`. Any subscriber always knows the device's current state.

Use `device.py` and `monitor.py` from the support files available on the e-Learning platform.

Run `monitor.py` in one terminal and `device.py` in another. Observe the `online` status message followed by periodic data readings.

Clean disconnect: Press `Ctrl+C` on `device.py`. The device itself publishes `offline` before disconnecting. `monitor.py` sees the transition immediately.

Ungraceful disconnect: Start `device.py` again. Find its PID with `ps aux | grep device.py` and kill it forcefully:

```
$ kill -9 <pid>
```

The TCP connection drops without a `DISCONNECT`. The broker detects the lost connection after the keepalive timeout (approximately 1.5 × 5 = 7 seconds) and publishes the LWT on the device's behalf. `monitor.py` receives `lost` from the broker — distinct from the `offline` the device sends on a clean shutdown.

```
device.py connects:
  CONNECT { will_topic="devices/sensor-01/status", will_payload="lost", will_retain=True }

on_connect fires:
  PUBLISH "devices/sensor-01/status" = "online" [retain]

... normal operation ...

kill -9: TCP drops, no DISCONNECT sent
  Broker detects keepalive timeout (~7 s)
  Broker PUBLISHES "devices/sensor-01/status" = "lost" [retain]  ← will fires
```

> **Thinking Question:** The LWT fires only on unexpected disconnects. A clean `DISCONNECT` packet suppresses it. For a device that reboots on a schedule, what must the application do before calling `disconnect()` to ensure the `offline` status is published?

> **Thinking Question:** A new `device.py` instance starts with the same `client_id` after the previous one was killed. The broker still has a retained `offline` message on `devices/sensor-01/status`. What does `monitor.py` see when the new instance connects and publishes `online`? In what order do the messages arrive, and why?

4.3 Coding Exercise: Fleet Monitor

Extend `device.py` to accept a device ID as a command-line argument:

```
DEVICE_ID = sys.argv[1]  # e.g. python3 device.py sensor-02
```

Run three instances simultaneously: `sensor-01`, `sensor-02`, `sensor-03`. Extend `monitor.py` to maintain a dictionary of device states and print a summary table every 10 seconds:

```
Device       Status    Last seen
-----------  --------  ---------
sensor-01    online    3s ago
sensor-02    online    1s ago
sensor-03    online    2s ago
```

Kill one device with `kill -9`. Observe the table update after the will fires (~7 seconds later).

This exercise combines topic wildcards from Part 2, QoS 1 for the status messages from Part 3, and the retained message and LWT mechanics from this part.

References

1. OASIS Standard. *MQTT Version 3.1.1*.
2. Eclipse Mosquitto. *Documentation*.
3. Eclipse Paho. *Python MQTT Client Documentation*.
4. HiveMQ. *MQTT Essentials*.
5. Eclipse Mosquitto. *mosquitto_pub(1)*.
6. Eclipse Mosquitto. *mosquitto_sub(1)*.